REMOVING TOKENIZATION BIAS: EXACT BYTE-LEVEL PREDICTION AND ENSEMBLING FROM TOKENIZED LANGUAGE MODELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Modern language models operate on tokens rather than bytes, and this discrete mapping induces a *tokenization bias*: the next-token distribution of a tokenized model can differ sharply from the byte-level distribution of the text it was trained on. The bias is most damaging when a prompt ends in the middle of a token, as in code completion and fill-in-the-middle (FIM), where the tokenizer's merge rules can make the correct next character impossible under the model, and it also makes models with different vocabularies hard to combine. We show that, without any retraining, a trained tokenized LM induces an equivalent byte-level process whose probabilities can be computed exactly. Our key observation is that the probability of a byte string equals the total probability of the *valid* token sequences that cover it; we prove this as a byte-token representation lemma, and give a reverse-search algorithm that finds these cover encodings and an inference-time procedure that samples the next byte, needing one model evaluation per byte for the canonical encoding of the current context. On synthetic Markov chains our method recovers the ground-truth transition probabilities that tokenized models get badly wrong. On the HumanEval random-span FIM benchmark with CodeLlama-7b, byte-level prediction raises the greedy pass rate in the suffix-prefix-middle template from 45.0 to 63.9 (above token healing at 62.7 and token alignment at 63.0), and beam search improves it further. Because byte probabilities live in a vocabulary-independent space, we can average the predictions of models with different tokenizers: top-2 ensembles improve over the best member on MMLU, PIQA, NQ, TriviaQA and HumanEval, but not when members differ greatly in ability (GSM8K). Single-model byte-level accuracy is mostly on par with token-level accuracy, with some small drops.

1 INTRODUCTION

Tokenization is a core component of current language models (LMs): text is compressed into sequences of subword tokens, typically with byte-pair encoding (Gage, 1994; Sennrich et al., 2016) or related schemes (Kudo & Richardson, 2018), and the LM is trained to predict the next token. Tokens shorten sequences and thus reduce computation, but they come at a price. The mapping from bytes to tokens is deterministic, so the model only ever sees one segmentation of any string, and what it learns is a distribution over *token* sequences rather than over the *bytes* that users actually care about. We call the resulting discrepancy between the token-level predictions and the underlying byte-level distribution *tokenization bias*.

Tokenization bias is easiest to see when a prompt ends in the middle of a token. Consider the completion prompt in Figure 1 (left), which ends in `return 5 + a`. A greedy tokenizer may have merged `aa` into a single token had the full word been present, so a model trained on tokenizer outputs never saw the segmentation "`a`" followed by "`a`" and assigns that continuation probability close to zero, even though the correct completion is exactly this one. This arises in practice in code completion and fill-in-the-middle (FIM) settings, where the cursor can fall anywhere inside a word, and in any input whose boundary does not align with the tokenizer's. Heuristic repairs such as token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) back up a few tokens and constrain generation to match the prompt, but they are not derived from the model's probabilistic

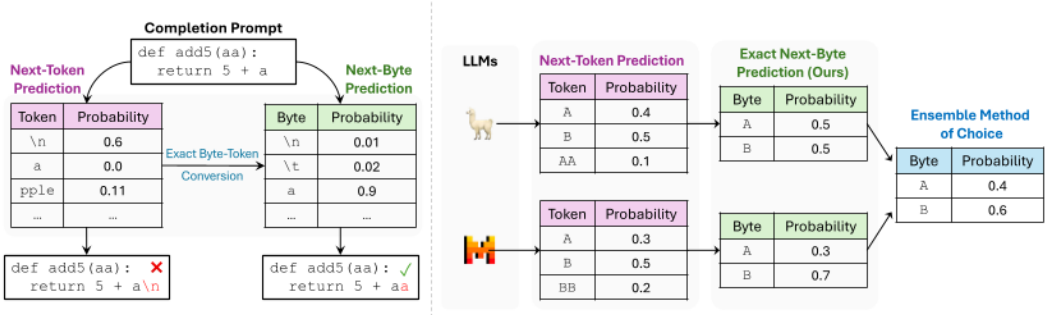


Figure 1: Left: tokenized LMs experience tokenization bias when prompts end mid-token, as in this code completion example, where the correct continuation "a" has zero probability of being chosen at the token level. Our byte-level prediction avoids the problem using the same model. Right: our method maps next-token predictions of arbitrary tokenized LMs to statistically equivalent next-byte predictions (Section 3), which enables any ensembling strategy, such as averaging or a mixture of experts.

semantics. A second consequence of token dependence is that two LMs with different vocabularies define distributions over disjoint output spaces, so their predictions cannot simply be averaged or mixed.

We hypothesize that, for any trained tokenized LM, there is a statistically equivalent byte-level process that can be recovered without retraining. The intuition is that tokenization bias is an artifact of *conditioning on a single, deterministic tokenization of the past*. If instead we account for all valid token sequences that can represent the byte string observed so far, the bias disappears. We make this precise and obtain an inference-time algorithm that turns the next-token logits of any tokenized LM into exact next-byte probabilities.

Our contributions are:

- **Theory.** We formalize tokenization bias, define valid encodings and cover encodings, and prove a byte-token representation lemma that expresses the probability of any byte string as a sum of token-sequence probabilities over its cover encodings (Section 3).
- **Algorithm.** We give a reverse search that finds cover encodings by tokenizing backwards from the end of the context, and a sampling procedure that uses them to draw the next byte, with no weight updates, effectively turning a tokenized model into a token-free one.
- **Robust FIM.** On the HumanEval random-span benchmark with CodeLlama-7b, byte-level prediction removes most of the degradation caused by the suffix-prefix-middle (SPM) template and outperforms token healing and token alignment (Section 4).
- **Vocabulary-agnostic ensembling.** Mapping models to the shared byte space lets us average the predictions of models with different tokenizers; we report where this helps and where it does not.

2 Related Work

Tokenization and its side effects. Subword tokenizers such as BPE (Gage, 1994; Sennrich et al., 2016) and SentencePiece (Kudo & Richardson, 2018) are standard in LMs (Radford et al., 2019; Touvron et al., 2023; Jiang et al., 2023a). Their drawbacks have been studied from several angles: BPE can be a suboptimal segmentation for pretraining (Bostrom & Durrett, 2020), tokenizer design choices affect efficiency and downstream behavior (Dagan et al., 2024), and the formal properties of tokenization, such as spurious ambiguity and inconsistency, have been analyzed by Gastaldi et al. (2024). Stochastic segmentation during training (Kudo, 2018; Provilkov et al., 2020) exposes the model to multiple segmentations of the same text, whereas we keep the trained model unchanged and fix the problem at inference.

Marginalizing over tokenizations. Because many token sequences decode to the same string, the probability of a string is properly the sum over its tokenizations. Cao & Rimell (2021) advocate evaluating LMs by marginal likelihood over tokenizations, Chirkova et al. (2023) study whether marginalizing helps in practice, and Geh et al. (2024) study non-canonical tokenizations of a text under LLMs. Related work on computing word- and character-level probabilities from subword LMs, motivated by psycholinguistic surprisal, treats the mismatch between token and character spans (Pimentel & Meister, 2024; Giulianelli et al., 2024). Our setting differs in that we exploit the determinism of the tokenizer: only *valid* encodings (those the tokenizer could produce) carry probability, which prunes the sum, and we use the resulting exact next-byte conditionals for *generation* and *ensembling*, rather than only for scoring.

Prompt-boundary heuristics and FIM. Fill-in-the-middle training lets autoregressive models infill code (Bavarian et al., 2022; Fried et al., 2023; Rozière et al., 2023), and the Prefix-Suffix-Middle (PSM) and Suffix-Prefix-Middle (SPM) templates are the two common ways to present a FIM prompt. SPM places the prefix last and can therefore end mid-token. Token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) mitigate partial-token prompts by backing up and constraining the continuation to be consistent with the prompt text. We compare against both and show that exact byte-level conditionals are at least as good in every setting we test.

Token-free models. Training directly on bytes or characters avoids tokenization altogether (Clark et al., 2022; Xue et al., 2022; Yu et al., 2023; Wang et al., 2024), at the cost of longer sequences and of retraining. Our approach instead derives a byte-level predictor from an already trained tokenized LM.

Model ensembling. Mixtures of experts (Shazeer et al., 2017; Jiang et al., 2024) combine subnetworks of a jointly trained model, while ensembles of independently trained LLMs either select or fuse complete outputs (Jiang et al., 2023b), distill models into one (Wan et al., 2024), or must reconcile vocabularies explicitly during decoding (Xu et al., 2024; Huang et al., 2024). Byte space is a common denominator that requires no vocabulary alignment: the output of any model is mapped to the same byte distribution and any ensembling rule can be applied.

3 METHODOLOGY

3.1 SETTING AND TOKENIZATION BIAS

Let $\mathcal{X}$ be the set of bytes and $\mathcal{V} \subset \mathcal{X}^+$ a token vocabulary, which we assume contains every single byte so that every string is encodable. We write $\mathrm{dec}(\mathbf{t})$ for the concatenation of the bytes of a token sequence $\mathbf{t} = (t_1, \ldots, t_k)$, and x_i^j for the byte substring from position i to j. A deterministic tokenizer (for example BPE) is a map $\mathrm{enc} : \mathcal{X}^* \to \mathcal{V}^*$ with $\mathrm{dec}(\mathrm{enc}(x)) = x$.

Definition 1 (Valid encoding)**.** *A token sequence* $\mathbf{t}$ *is* valid *if* $\mathrm{enc}(\mathrm{dec}(\mathbf{t})) = \mathbf{t}$*, that is, if it is exactly what the tokenizer would output for the bytes it represents. Otherwise it is* invalid*.*

For BPE, a token sequence in which two adjacent tokens would have been merged into a longer token by the merge rules is invalid. Since the training data of a tokenized LM are produced by enc, the LM $P(t_{i+1} \mid t_1^i)$ is trained on valid encodings only, and a well-trained model should put (almost) no mass on invalid ones. We write $P(\mathbf{t}) = \prod_i P(t_i \mid t_1^{i-1})$ for the probability of a token sequence (conditioning on any special start token is left implicit). Our working assumption is that the LM defines a distribution over valid token sequences; the byte-level process it induces is

$$P(x_1^n) \;:=\; \Pr\left[\mathrm{dec}(T) \text{ begins with } x_1^n\right], \qquad T \sim P, \tag{1}$$

and the object we want to sample from is $P(x_{n+1} \mid x_1^n) = P(x_1^{n+1})/P(x_1^n)$.

The standard practice instead encodes the context as $\mathbf{t}^* = \mathrm{enc}(x_1^n)$ and reads off the next-token distribution $P(\cdot \mid \mathbf{t}^*)$; the implied next-byte distribution is $P_{\mathrm{tok}}(b \mid x_1^n) = \sum_{v \in \mathcal{V}:\, v_1 = b} P(v \mid \mathbf{t}^*)$ (up to normalization), where v_1 is the first byte of token v. This conditions on the tokenization $\mathbf{t}^*$ in which a token boundary falls exactly after byte n. If the true continuation would have merged byte n with what follows, then $\mathbf{t}^*$ followed by that continuation is an invalid encoding and receives probability zero. We call the gap between P_{tok} and $P(x_{n+1} \mid x_1^n)$ *tokenization bias*.

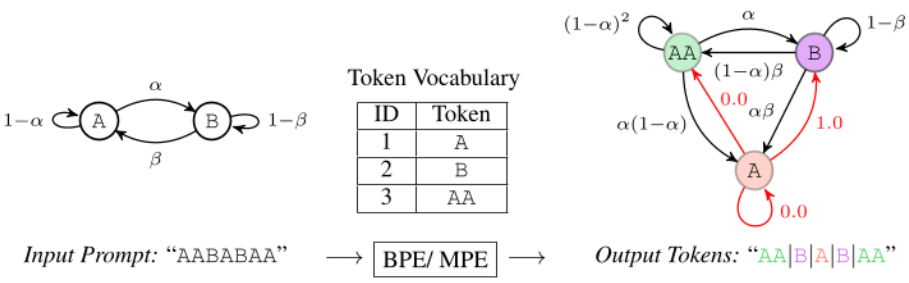


Figure 2: Tokenization bias on a first-order Markov chain over bytes A and B with a BPE vocabulary that merges AA. Given the context token "A", the tokenized model never samples the next token A (red probabilities), whereas the true byte-level probability is $1-\alpha$. In practice this bias occurs when prompts end with a token that is part of another token, a common issue in FIM tasks.

A minimal example. Figure 2 shows a first-order Markov chain over bytes $\{\texttt{A}, \texttt{B}\}$ with $P(\texttt{A} \mid \texttt{A}) = 1 - \alpha$, and a BPE vocabulary $\{\texttt{A}, \texttt{B}, \texttt{AA}\}$. After the token A, the next token can never be A or AA, since the tokenizer would have merged the two bytes into the single token AA. A model trained on tokenized samples therefore assigns zero probability to A after A and probability one to B, whereas the true byte-level probability of A is $1 - \alpha$. The mass is not lost: it is carried by the other encoding of the same context in which the A is part of the token AA.

3.2 Cover encodings and the byte-token representation lemma

To account for *all* the ways a byte string can be represented, we use the set of valid encodings that first reach or pass its end.

Definition 2 (Cover encodings). *For a byte string x_1^n, $\mathrm{cover}(x_1^n)$ is the set of valid token sequences $\mathbf{t} = (t_1, \ldots, t_k)$ such that (i) $\mathrm{dec}(\mathbf{t})$ begins with x_1^n, and (ii) $|\mathrm{dec}(t_1^{k-1})| < n$, i.e. the last token t_k is the one that covers byte n.*

Figure 3 (left) illustrates $\mathrm{cover}(x_1^n)$ for a small BPE vocabulary: its elements differ in where the last token starts and how far it extends past byte n, and invalid sequences are excluded.

Lemma 1 (Byte-token representation). *If P is supported on valid token sequences, then for every byte string x_1^n*

$$P(x_1^n) \;=\; \sum_{\mathbf{t} \in \mathrm{cover}(x_1^n)} P(\mathbf{t}). \tag{2}$$

Proof. Let T be a random token sequence drawn from P and, if $\mathrm{dec}(T)$ begins with x_1^n, let k be the smallest index with $|\mathrm{dec}(T_1^k)| \geq n$. Then T_1^k satisfies conditions (i) and (ii) and is valid, hence $T_1^k \in \mathrm{cover}(x_1^n)$. Conversely, if $T_1^k \in \mathrm{cover}(x_1^n)$ then $\mathrm{dec}(T)$ begins with x_1^n. The set $\mathrm{cover}(x_1^n)$ is prefix-free: if $\mathbf{t}$ were a proper prefix of $\mathbf{t}'$, both in the cover, then $|\mathrm{dec}(\mathbf{t})| \geq n$ would contradict condition (ii) for $\mathbf{t}'$. The events $\{T_1^k = \mathbf{t}\}$, $\mathbf{t} \in \mathrm{cover}(x_1^n)$, are therefore disjoint and their union is the event $\{\mathrm{dec}(T) \text{ begins with } x_1^n\}$. Since $\Pr[T_1^k = \mathbf{t}] = P(\mathbf{t})$, the claim follows. □

Applied to the example above, $\mathrm{cover}(\texttt{A}) = \{(\texttt{A}), (\texttt{AA})\}$, and $P(\texttt{AA})/P(\texttt{A})$ computed through the lemma recovers the chain's $1 - \alpha$ rather than zero. By the chain rule, $\prod_n P(x_{n+1} \mid x_1^n)$ equals the probability the LM assigns to the string, so sampling bytes from these conditionals is equivalent to sampling from the LM and decoding. Lemma 1 thus turns next-byte prediction into a search problem: find $\mathrm{cover}(x_1^n)$ and read off the token-level probabilities.

3.3 Reverse search for cover encodings

Enumerating all token sequences that decode to a prefix of the context is exponential. Instead, we construct cover encodings *backwards* (Algorithm 1, Figure 3 right). The last token of an element of $\mathrm{cover}(x_1^n)$ starts at some position $s = n - i$ and must have x_s^n as a prefix. For each start position we

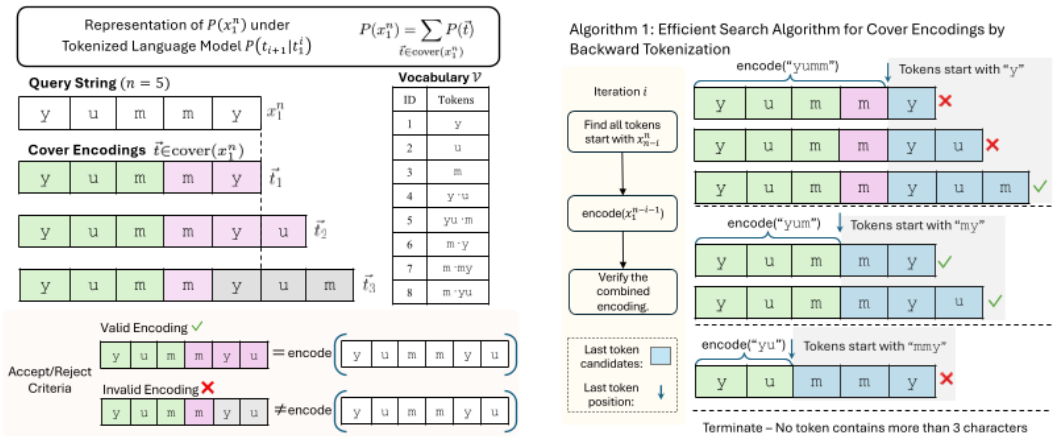


Figure 3: Left: representation of $P(x_1^n)$ with a tokenized LM, with an example of cover encodings and of a valid and an invalid encoding (Definition 1). Right: illustration of Algorithm 1, the search for cover encodings. Green ticks and red crosses denote valid and invalid encodings. The termination step is easy to implement though not drawn. BPE is used in this example.

Algorithm 1 Reverse search for cover encodings

Require: byte string x_1^n, vocabulary $\mathcal{V}$, tokenizer enc, maximum token length L

$\mathcal{C} \leftarrow \emptyset$
for $i = 0, 1, \ldots, \min(n, L) - 1$ **do**
  $s \leftarrow n - i$ ▷ start position of the last token
  $\mathbf{t}_{\text{pre}} \leftarrow \text{enc}(x_1^{s-1})$
  for every token $v \in \mathcal{V}$ that has x_s^n as a prefix **do**
    $\mathbf{t} \leftarrow \mathbf{t}_{\text{pre}} \circ v$
    if $\text{enc}(\text{dec}(\mathbf{t})) = \mathbf{t}$ **then** ▷ valid encoding
      $\mathcal{C} \leftarrow \mathcal{C} \cup \{\mathbf{t}\}$
    end if
  end for
end for
return $\mathcal{C}$

find all such tokens v in the vocabulary (for instance with a trie), prepend the tokenizer's encoding of the preceding bytes x_1^{s-1}, and keep the candidate if the combined sequence is valid in the sense of Definition 1. The search stops once x_s^n is longer than the longest token in $\mathcal{V}$, so at most $L = \max_v |v|$ start positions are examined and $|\text{cover}(x_1^n)|$ is bounded by the number of (start position, token) pairs.

Every returned sequence is valid by construction, and satisfies Definition 2: it decodes to a string that begins with x_1^n and its last token alone covers byte n. Completeness relies on a property of the tokenizer: that the tokens preceding the last one in a valid encoding are the canonical encoding $\text{enc}(x_1^{s-1})$ of the bytes they represent. This is the property the search is built on, and we do not prove it for arbitrary tokenizers here. Each cover element needs its probability $P(\mathbf{t})$, which is obtained with the LM; the cost is paid once for the prompt, and afterwards the elements are updated incrementally as described next.

3.4 Sampling the next byte

Suppose we hold $\mathcal{C}_n = \text{cover}(x_1^n)$ together with the weights $w(\mathbf{t}) = P(\mathbf{t})$. The next-byte distribution is $P(x_{n+1} = b \mid x_1^n) \propto P(x_1^n b) = \sum_{\mathbf{t} \in \text{cover}(x_1^n b)} P(\mathbf{t})$ by Lemma 1, and $\text{cover}(x_1^n b)$ is assembled from $\mathcal{C}_n$ without a new search (Algorithm 2, Figure 4):

Algorithm 2 One step of byte-level sampling

Require: context x_1^n, cover set $\mathcal{C}_n$ with weights $w(\mathbf{t}) = P(\mathbf{t})$, tokenized LM P

$\mathbf{t}^* \leftarrow \mathrm{enc}(x_1^n)$; query the LM once for $P(\cdot \mid \mathbf{t}^*)$
for every byte $b \in \mathcal{X}$ **do**
  $\mathcal{C}(b) \leftarrow \{\mathbf{t} \in \mathcal{C}_n : |\mathrm{dec}(\mathbf{t})| > n,\ \mathrm{dec}(\mathbf{t})_{n+1} = b\}$ ▷ regrouping
  for every token $v \in \mathcal{V}$ whose first byte is b **do**
    if $\mathrm{enc}(\mathrm{dec}(\mathbf{t}^* \circ v)) = \mathbf{t}^* \circ v$ **then** ▷ extension, accept if valid
      add $\mathbf{t}^* \circ v$ to $\mathcal{C}(b)$ with weight $w(\mathbf{t}^*)\, P(v \mid \mathbf{t}^*)$
    end if
  end for
  $\mathrm{score}(b) \leftarrow \sum_{\mathbf{t} \in \mathcal{C}(b)} w(\mathbf{t})$
end for
$p(b) \leftarrow \mathrm{score}(b) / \sum_{b'} \mathrm{score}(b')$; draw $x_{n+1} \sim p$ (or take the argmax)
return x_{n+1} and $\mathcal{C}_{n+1} \leftarrow \mathcal{C}(x_{n+1})$

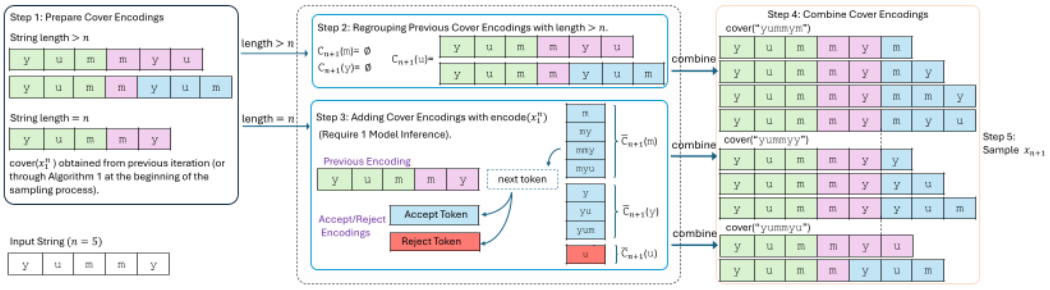


Figure 4: Sampling the next byte x_{n+1} from a tokenized LM $P(t_{k+1} \mid t_1^k)$, continuing the example of Figure 3. Previous cover encodings that extend beyond n are regrouped by their next byte (Step 2); the canonical encoding of x_1^n is extended with every token that is accepted as valid (Step 3, one model evaluation); the groups are combined per byte (Step 4) and the next byte is sampled (Step 5). $P(\mathbf{t})$ is omitted for simplicity.

1. *Regrouping.* Elements $\mathbf{t} \in \mathcal{C}_n$ with $|\mathrm{dec}(\mathbf{t})| > n$ already contain byte $n+1$. They belong to $\mathrm{cover}(x_1^n b)$ for $b = \mathrm{dec}(\mathbf{t})_{n+1}$, keep their weights, and need no model call.
2. *Extension.* The only element of $\mathcal{C}_n$ that decodes to exactly x_1^n is $\mathbf{t}^* = \mathrm{enc}(x_1^n)$, since valid encodings are unique. One model evaluation gives $P(v \mid \mathbf{t}^*)$ for all $v \in \mathcal{V}$. For every token v with first byte b such that $\mathbf{t}^* \circ v$ is valid, we add $\mathbf{t}^* \circ v$ to $\mathrm{cover}(x_1^n b)$ with weight $w(\mathbf{t}^*)P(v \mid \mathbf{t}^*)$; invalid extensions are rejected.

Summing the weights per byte, normalizing over bytes, and sampling (or taking the argmax) yields x_{n+1}; the chosen group becomes $\mathcal{C}_{n+1}$. This is the aggregation of the idea in the introduction: the probability of the next byte is the total probability of the next tokens that start with that byte, conditioned on every valid history and not just on $\mathbf{t}^*$. In Figure 1 (right), a model that gives 0.4, 0.5, 0.1 to the tokens `A`, `B`, `AA` gives byte `A` the probability $0.4 + 0.1 = 0.5$. Normalization over bytes means that probability mass that the model places on rejected (invalid) tokens is removed rather than redistributed to a particular token, so the conditional is defined with respect to valid encodings only. In total, each generated byte costs one forward pass of the LM on the canonical encoding of the current context, plus tokenizer calls for validity checks.

Prompts ending mid-token. The initial cover set of the prompt is obtained from Algorithm 1. If the prompt ends mid-token, the cover set contains encodings in which the final prompt bytes are the beginning of a longer token, so the next byte can be completed within that token. This is the situation of Figure 1 (left): the encoding in which `aa` is a single token supplies the probability of the second `a` that the canonical encoding of the prompt cannot. For FIM prompts we treat the special sentinel tokens as fixed tokens of the context and apply the byte-level treatment to the text.

3.5 BEAM SEARCH AND ENSEMBLING IN BYTE SPACE

Since the procedure returns an exact conditional distribution over bytes, any decoding rule over bytes can be used. We use greedy decoding, temperature sampling, and beam search over byte sequences, where a beam width of one coincides with greedy decoding.

For M tokenized LMs with arbitrary, possibly different, vocabularies, each model m maintains its own cover set and yields its own next-byte distribution $P_m(\cdot \mid x_1^n)$. The ensemble distribution is

$$P_{\text{ens}}(b \mid x_1^n) = \sum_{m=1}^{M} \lambda_m P_m(b \mid x_1^n), \qquad \lambda_m \geq 0,\ \sum_m \lambda_m = 1, \tag{3}$$

and in our experiments we use uniform weights $\lambda_m = 1/M$. The sampled byte is fed back to all members, which advance in lockstep. Figure 1 (right) shows a two-model example in which byte distributions $(0.5, 0.5)$ and $(0.3, 0.7)$ are averaged to $(0.4, 0.6)$. Other combiners, such as a learned mixture of experts, would operate on the same byte distributions.

4 EXPERIMENTS

We evaluate the method on (i) synthetic Markov chains where the ground truth is known, (ii) code infilling with prompts that may end mid-token, and (iii) ensembling of models with different vocabularies on knowledge, reasoning and coding benchmarks. In all experiments the model weights are untouched: token-level and byte-level results come from the same model, and differ only in how the next-symbol distribution is computed.

4.1 SETUP

Models. All models are Transformer LLMs. General-purpose models: Llama2-7B (Touvron et al., 2023), Yi-1.5-6B (Young et al., 2024) and Mistral-7B-v0.3 (Jiang et al., 2023a). Code models: CodeLlama-7b and CodeLlama-13b (Rozière et al., 2023) and Yi-Coder-1.5B. For synthetic experiments we use first- and third-order Markov chains and a GPT-2-style model (Radford et al., 2019) with six hidden layers trained on synthetic data.

Benchmarks and metrics. Code infilling uses the HumanEval random-span FIM benchmark of Bavarian et al. (2022) (Python), in both the PSM and SPM templates, reporting greedy pass rate and pass@10 (the maximum over a temperature sweep). Knowledge and reasoning: MMLU (Hendrycks et al., 2021), PIQA (Bisk et al., 2020), Natural Questions (NQ) (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017) and GSM8K (Cobbe et al., 2021), reported as accuracy. Code generation: HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), reported as pass@1.

Baselines. *Token-level* is the model's default autoregressive prediction. *Token healing* (Dagan et al., 2024) and *token alignment* (Athiwaratkun et al., 2024) are heuristics for prompt boundaries, evaluated for SPM mode only. For ensembles, we compare to a *top-2 voting* baseline. Ensembles combine the two best-performing models on the respective benchmark by averaging byte-level probabilities (Section 3.5). We run single models with token-level and byte-level predictions. We report a single number per configuration without confidence intervals, so differences of a few tenths of a point should not be over-interpreted.

4.2 VERIFYING TOKENIZATION BIAS ON SYNTHETIC DATA

In the first-order chain of Figure 2, the token-level model assigns probability 0.0 to the next token `A` after the token `A`, while the ground-truth probability of the byte `A` is $1 - \alpha$. For a third-order chain, we train the GPT-2-style model on tokenized samples and compare the next-byte probabilities with the ground truth $P(x_{n+1} \mid x_{n-2}^n)$ (Figure 5). The tokenized model shows severe bias; in the figure its estimates are close to zero for most three-byte contexts. Applying our byte-token conversion recovers the ground truth transition probabilities, with the corrected estimates visually matching the ground truth in every context shown.

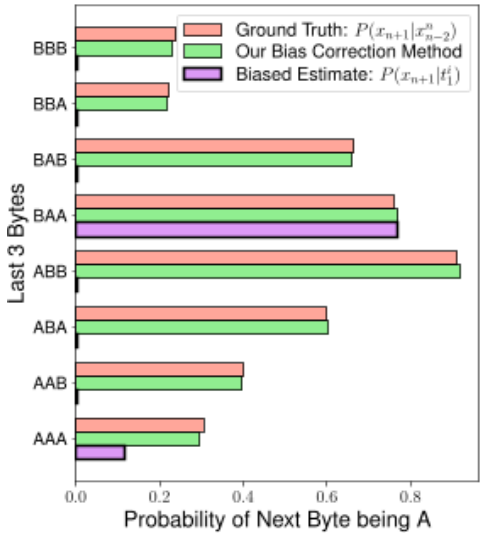


Figure 5: Tokenization bias on a third-order Markov chain. The biased token-level estimate $P(x_{n+1} \mid t_1^k)$ deviates strongly from the ground truth, while our byte-token conversion (bias correction, Section 3) accurately recovers $P(x_{n+1} \mid x_{n-2}^n)$ of the original chain.

	PSM		SPM	
Predictions	Greedy	@10	Greedy	@10
Token-level	64.9	84.0	45.0	66.5
+ healing	|	|	62.7	83.8
+ alignment	|	|	63.0	82.9
Byte-level	**65.7**	**84.1**	**63.9**	**84.3**

@10 here is the max over the temperature sweep on the right

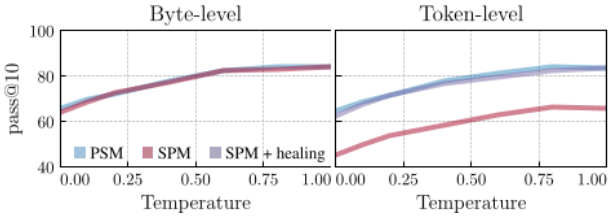


Figure 6: Pass rates for the HumanEval random-span benchmark of Bavarian et al. (2022) with CodeLlama-7b for Python infilling. Byte-level predictions give better completions in all settings. We evaluate the PSM and SPM templates, with token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) as baselines. The SPM template forces the model to predict from the middle of a token, which byte-level prediction handles. Right: pass@10 as a function of sampling temperature; the table reports the maximum over this sweep.

4.3 Code infilling with prompts that end mid-token

Figure 6 reports pass rates on the HumanEval random-span benchmark with CodeLlama-7b. The numbers in the table are:

- **PSM:** token-level greedy / pass@10 are 64.9 / 84.0, and byte-level are 65.7 / 84.1.
- **SPM:** token-level 45.0 / 66.5; token healing 62.7 / 83.8; token alignment 63.0 / 82.9; byte-level 63.9 / 84.3.

SPM degradation. With standard token-level prediction, switching from PSM to SPM reduces the greedy pass rate from 64.9 to 45.0 and pass@10 from 84.0 to 66.5. This drop is consistent with the explanation that SPM prompts end mid-token and that the resulting segmentation is out-of-distribution for the model. Byte-level prediction removes most of the gap: SPM greedy is 63.9, close to PSM, an improvement of 18.9 points (absolute) over the token-level SPM baseline. As the right panel of Figure 6 shows, the byte-level SPM and PSM curves are nearly indistinguishable across temperatures, while the token-level SPM curve lies far below the others.

Comparison to heuristics. Byte-level prediction is the best method in every SPM column: 63.9 greedy against 62.7 (healing) and 63.0 (alignment), and 84.3 pass@10 against 83.8 and 82.9. The

Table 1: Pass rate on the HumanEval FIM task with byte-level prediction as a function of beam width (CodeLlama-7b). Width 1 is greedy decoding.

Beam width	PSM	SPM
1	65.7	63.9
2	66.8	65.9
4	**67.7**	**66.8**

Table 2: Accuracy of open-source LMs on knowledge and reasoning benchmarks with token-level (T) and byte-level (B) prediction. Voting and ensemble rows use the top-2 performing models, in byte space; "–" means not applicable or not evaluated. Voting is reported only for MMLU and PIQA. Best result per benchmark in bold.

	MMLU		PIQA		NQ		TriviaQA		GSM8K	
Model	T	B	T	B	T	B	T	B	T	B
Llama2-7B	45.4	45.7	78.1	78.2	25.0	23.4	58.4	58.1	15.1	12.5
Yi-1.5-6B	63.4	63.4	78.5	78.5	22.8	22.7	53.7	53.4	61.3	**61.5**
Mistral-7B-v0.3	62.1	62.1	80.1	80.3	28.5	28.8	63.6	63.5	39.0	39.0
Voting (top-2)	–	62.1	–	80.1	–	–	–	–	–	–
Top-2 ensemble (ours)	–	**65.4**	–	**80.7**	–	**30.0**	–	**64.2**	–	55.8

margins over the heuristics are small (about one point in greedy, about 0.5 to 1.4 points in pass@10), but the byte-level approach has the advantage of being derived from the model's distribution rather than being a fix tailored to this failure.

Gains in PSM. Surprisingly, byte-level prediction also improves the greedy pass rate in the "safe" PSM mode (65.7 versus 64.9), where prompt boundaries are largely not an issue; pass@10 is essentially tied (84.1 versus 84.0). We did not investigate the cause, and the PSM difference is small and we have no variance estimate. One possible explanation, which we have not tested, is that tokenization bias also affects the tokens generated within the infilled span.

Qualitative failures. We inspected cases where the prompt ends in the middle of a word, for instance in the variable names "delimeter", "collatz" and "palindrome". In these cases the standard model failed to complete the word because of the mismatch with token boundaries, while the byte-level method identified the correct completion (Appendix A).

Beam search. Because byte-level predictions are exact conditionals, they can be combined with beam search. Table 1 shows that increasing the beam width from 1 to 4 improves the FIM pass rate monotonically in both templates, by 2.0 points in PSM (65.7 to 67.7) and 2.9 points in SPM (63.9 to 66.8). In both templates the first doubling of the width (1 to 2) gives the larger gain (1.1 points in PSM, 2.0 in SPM) and the second (2 to 4) a smaller one (0.9 in both).

4.4 Byte-level prediction and ensembling on general benchmarks

Tables 2 and 3 compare token-level and byte-level predictions of individual models and the byte-level top-2 ensembles. Ensembles are formed from models with different vocabularies by averaging in byte space.

Single models: byte-level prediction is mostly on par, with some drops. Because the byte-level distribution is an exact reformulation of the model's distribution, we do not expect it to change accuracy much, and in most cells it does not: on MMLU, PIQA, NQ and TriviaQA the byte-level and token-level scores of Yi-1.5-6B and Mistral-7B-v0.3 differ by at most 0.3 points. Byte-level prediction is, however, not uniformly better. Of the 15 model–benchmark pairs in Table 2, 5 improve slightly, 4 are unchanged and 6 are lower, and the largest drops are for Llama2-7B (NQ: $25.0 \rightarrow 23.4$, GSM8K: $15.1 \rightarrow 12.5$). On HumanEval pass@1 the byte-level score is lower for all three code models (by 1.9 to 3.6 points), while on MBPP it is higher for CodeLlama-7b ($40.6 \rightarrow 42.4$)

Table 3: Pass@1 of code LMs on HumanEval and MBPP with token-level (T) and byte-level (B) prediction. The ensemble uses the top-2 models in byte space. For HumanEval these are Yi-Coder-1.5B and CodeLlama-13b. Best result per benchmark in bold.

	HumanEval @1		MBPP @1	
Model	T	B	T	B
CodeLlama-7b	32.3	28.7	40.6	42.4
CodeLlama-13b	35.8	33.5	47.6	47.4
Yi-Coder-1.5B	38.4	36.5	52.8	**53.6**
Top-2 ensemble (ours)	–	**42.1**	–	**53.6**

and Yi-Coder-1.5B ($52.8 \rightarrow 53.6$) and about equal for CodeLlama-13b. We have not isolated the cause of these drops. Greedy decoding over bytes and over tokens are different procedures and can pick different continuations even when the two distributions are equivalent, and the model may place some probability on invalid encodings that we reject, but we did not test these explanations.

Ensembles of models with different vocabularies. The ensemble of the top-2 models is able to combine models with distinct tokenizers (for example Yi, Mistral and Llama) with no vocabulary alignment. It exceeds the best individual model on MMLU (65.4 versus 63.4 for the best token-level model), PIQA (80.7 versus 80.3), NQ (30.0 versus 28.8), TriviaQA (64.2 versus 63.6) and HumanEval (42.1 versus 38.4, an improvement of 3.7 points over the best single model, using Yi-Coder-1.5B and CodeLlama-13b). On MBPP, the ensemble (53.6) matches the best byte-level single model and is 0.8 points above the best token-level single model. The top-2 voting baseline does not improve over its best member (62.1 on MMLU, below the 63.4 of Yi-1.5-6B, and 80.1 on PIQA), whereas probability averaging yields gains of 3.3 and 0.6 points over voting on these two benchmarks.

Limitation: divergent members. Ensembling does not help when the members differ too much in ability. On GSM8K, Yi-1.5-6B (61.5 byte-level) outperforms Mistral-7B-v0.3 (39.0) by more than 20 points, and the uniform-weight ensemble reaches only 55.8, below the best single model. This suggests that unweighted averaging gives the weaker model too much influence; we did not try weighted combinations, which would be a natural next step.

5 Conclusion

We identified tokenization bias, the discrepancy between the token-level predictions of an LM and the byte-level distribution of the text it models, and showed that it can be removed at inference time. By proving that the probability of a byte string equals the total probability of its valid cover encodings, and by searching for those encodings backwards from the end of the context, we convert the next-token predictions of any tokenized LM into exact next-byte predictions without retraining. The approach recovers ground-truth probabilities on synthetic Markov chains, closes most of the SPM–PSM gap in code infilling and slightly exceeds token healing and token alignment, and permits ensembling of models with different vocabularies.

The method has limitations. Single-model byte-level accuracy is not always higher than token-level accuracy: we observed small drops on HumanEval and for Llama2-7B on NQ and GSM8K. Uniform averaging fails to help when members differ greatly in capability (GSM8K). The completeness of the reverse search relies on a property of the tokenizer that we state but do not prove in general, and we report single numbers without confidence intervals; we also report no wall-clock or memory measurements, so the practical overhead of maintaining cover sets and checking validity remains to be quantified. Future directions include weighted or learned combinations of byte-level predictions, other ensembling rules such as mixtures of experts, and an analysis of why byte-level greedy decoding sometimes differs from token-level decoding.

REFERENCES

Ben Athiwaratkun, Shiqi Wang, Mingyue Shang, Yuchen Tian, Zijian Wang, Sujan Kumar Gonugondla, Sanjay Krishna Gouda, Robert Kwiatkowski, Ramesh Nallapati, Parminder Bhatia, and Bing Xiang. Token alignment via character matching for subword completion. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.

Kris Cao and Laura Rimell. You should evaluate your language model on marginal likelihood over tokenisations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Nadezhda Chirkova, Germán Kruszewski, Jos Rozen, and Marc Dymetman. Should you marginalize over possible tokenizations? In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2023.

Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10, 2022.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. Getting the most out of your tokenizer for pre-training and domain adaptation. *arXiv preprint arXiv:2402.01035*, 2024.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. InCoder: A generative model for code infilling and synthesis. In *International Conference on Learning Representations*, 2023.

Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.

Juan Luis Gastaldi, John Terilla, Luca Malagutti, Brian DuSell, Tim Vieira, and Ryan Cotterell. The foundations of tokenization: Statistical and computational concerns. *arXiv preprint arXiv:2407.11606*, 2024.

Renato Lui Geh, Honghua Zhang, Kareem Ahmed, Benjie Wang, and Guy Van den Broeck. Where is the signal in tokenization space? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.

Mario Giulianelli, Luca Malagutti, Juan Luis Gastaldi, Brian DuSell, Tim Vieira, and Ryan Cotterell. On the proper treatment of tokenization in psycholinguistics. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

Yichong Huang, Xiaocheng Feng, Baohang Li, Yang Xiang, Hui Wang, Bing Qin, and Ting Liu. Ensemble learning for heterogeneous large language models with deep parallel collaboration. In *Advances in Neural Information Processing Systems*, 2024.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023a.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023b.

Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 2017.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, 2018.

Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2018.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7, 2019.

Tiago Pimentel and Clara Meister. How to compute the probability of a word. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. BPE-Dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge fusion of large language models. In *International Conference on Learning Representations*, 2024.

Junxiong Wang, Tushaar Gangavarapu, Jing Nathan Yan, and Alexander M. Rush. MambaByte: Token-free selective state space model. *arXiv preprint arXiv:2401.13660*, 2024.

Yangyifan Xu, Jinliang Lu, and Jiajun Zhang. Bridging the gap between different vocabularies for LLM ensemble. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2024.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10, 2022.

Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, et al. Yi: Open foundation models by 01.AI. *arXiv preprint arXiv:2403.04652*, 2024.

Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Advances in Neural Information Processing Systems*, 2023.

A Additional notes on the FIM error analysis

We examined specific failures of the token-level model in the infilling benchmark in which the prompt ends in the middle of a word, namely the identifiers “delimeter”, “collatz” and “palindrome”. In these examples the prompt is cut inside the identifier, so the tokenizer would have segmented the full word differently from the truncated prompt. The token-level model failed to complete the word correctly because of this token-boundary mismatch, while the byte-level method correctly identified the completion. We only analyzed these cases qualitatively and do not report aggregate statistics on them.